Data-Driven Fleet Operations: Demand Analytics and Smart Charging for Urban Ride-Hailing in Chicago

Advanced Analytics and Applications

Team 06

Authors:
Louis Schreier (7381065)
Daniel Schumacher (7409287)
Timo Ercole (7408459)
Contact: lschreie@smail.uni-koeln.de

Supervisor: Univ.-Prof. Dr. Wolfgang Ketter
Co-Supervisor: Janik Muires

Chair of Information Systems for Sustainable Society
Faculty of Management, Economics and Social Sciences
University of Cologne

2026-07-26

Table of contents

List of Figures

1 Introduction

1.1 Business Context

The following report outlines the analysis of Bane & Wayne Partners (BWP) concerning the dynamics in the market of ride-hailing services with EV fleets exemplified at the US City of Chicago. The Data Science Team worked on the case study in order to provide data-driven insights and guidance for the objective to establish a platform for ride-hailing mobility services (data mining goal). Therefore, historical data is utilized to characterize demand, build initial models to forecast demand and designed a cost-efficient charging strategy as an agent. The business goal is to provide actionable intelligence and provide a clear path towards key criteria in launching and running the fleet platform.

1. **Where and when is demand concentrated, and where is it actually profitable to serve?** This drives market entry, fleet positioning, and driver scheduling.
2. **How much better than a simple seasonal schedule can demand be forecast?** Better predictions cut idle time and energy waste, and raise driver earnings.
3. **How should the fleet charge to minimize cost and the risk of an under-charged vehicles?** This provides implications for charging infrastructure.
4. **Is each analytical model worth what it costs to build, run, and maintain, or does a simpler baseline already captures most of the value?** Each model, is evaluated against this background.

2 Data Description

The analysis draws on three complementary data sources: Chicago taxi trip records for the demand signal, hourly weather data, and OpenStreetMap points of interest for the urban structure that shapes where trips originate. Full field-level data dictionaries are provided in Appendix A.

Table 1: The three data sources behind the analysis.

Source	Period	Volume	Spatial grain	Role
Taxi trips (Chicago Data Portal)	Jan 2024 – May 2026	14,985,679 trips, 21 fields	Census tract (~800 tracts)	Demand target, revenue
Weather (Open-Meteo, 2024)	Jan 2024 – May 2026	61,335 hourly obs.	3 zones (downtown, O'Hare, South Side)	Exogenous demand driver
Points of interest (Boeing, 2017)	static	9,124 locations, 7 categories	Point coordinates	Structural demand driver

Each trip of the taxi-dataset is described across 21 spatio-temporal and economic feature columns. All timestamps of these trips, are rounded to the nearest 15 minutes, which limits temporal precision but leaves the hourly-and-coarser aggregations this report relies on unaffected. Most consequentially, pickup and dropoff locations are reported only at census-tract level rather than as GPS coordinates, a resolution the analysis works around but never fully escapes (see Limitations, Section 6.4).

Weather is sampled at three zones rather than one, since Chicago's ~600 km² footprint shows real microclimatic differences between the lakefront, the dense core, and the far northwest that a single station would miss. Each observation carries nine meteorological variables that broadly describe temperature, precipitation and snow, wind, and cloud cover. Points of interest are static: they describe what is near a location, not what happened at a given time, and enter the models purely as a proxy for how strongly a hexagon attracts trips. They represent tag groups like, amenities, shops, tourism, and leisure, which incorporate fixed features of the city such as restaurants, transit stops and hospitals.

3 Data Preparation

3.1 Cleaning the Taxi Trip Data

Public portal data requires careful validation before it can be relied upon. This covered missing fields and their imputation, recordings with zero duration or zero distance, and trips with implausible speeds or fares. The complete data preparation rules are reported in Appendix B. Taken together, these steps remove just over 10 % of the raw records without distorting the underlying geography, as the per-neighborhood pickup counts before and after cleaning correlate at 1.0. Not every following analysis step calls for the same degree of rigor. Since partially faulty trips still represent genuine demand, we distinguish two cleaning levels. For demand forecasting utilized by prediction models, a minimal level of cleaning retains 99.99 % of rows, whereas trip-level spatial analytics only keeps 89.99%.

The weather and POI data required comparatively light preparation. The three weather zones were obtained by running K-means clustering over all trip pickup coordinates, which placed the zone centroids where demand is densest. Namely the downtown core, O'Hare, and the South Side. Beyond this, we dropped a redundant weather condition code and checked for gaps that could appear becasue of summer/winter time changes. The points of interest posed a different problem, as the raw OpenStreetMap tags were too fine-grained and overlapping, to be used directly. We therefore collapsed them into seven demand-relevant categories ranging from food and nightlife to transport, and reduced each geometry to a single coordinate. Aggregated per hexagon as category counts, these then serve as a proxy for how strongly each cell attracts trips (details in Appendix B).

3.2 From Trips to a Forecasting Grid

The forecasting models do not operate on individual trips but on an aggregated grid, with one row per (hexagon, time bucket) and the trip pickup count as the target variable. Cells that record no pickups are retained as explicit zero-demand rows, ensuring that the models also learn from low-demand periods rather than only from high-demand periods. Choosing the resolution of this grid is a performance trade-off, since finer cells localize demand more precisely, but with a larger share of empty cells. This leads to longer trainings and higher computational resources required. We account for this trade-off by founding our analysis on **H3 resolution 6 with 4-hour buckets**, which results in 27 hexagons $\times$ 5,107 time buckets = 137,889 rows at a manageable 12.6 % zero-demand cells. Moving one resolution higher would increase that share toward 50 % without a significant gain in operational precision (details in Appendix B, Table 11). The features comprise the calendar context (hour, day of week, month, weekend flags, US-holiday flags, time encoded as sine-cosine pairs with 23:00 positioned next to 00:00), the nine weather variables origin from the nearest zone, and a POI indicator, giving 51 input features per grid cell in total.

4 Descriptive Analytics

4.1 Temporal Map

In order to derive business implications a spatio-temporal view is created and analyzed in multiple dimensions. Taxi demand in Chicago follows predictable rhythms that are tied to work schedules, social activity, and weather, and understanding these rhythms is the natural starting point for any decision about fleet positioning and driver scheduling.

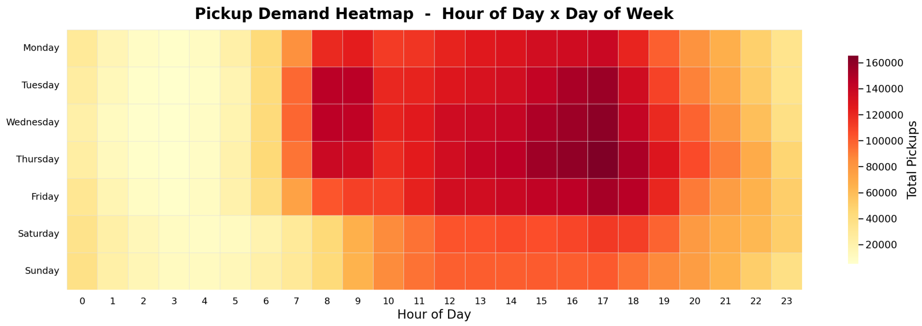

Figure 1: Pickup demand heatmap cross-tabulated by hour of day and day of week.

Weekday demand is distinctly bimodal, with a morning commuter peak around 08:00 followed by a stronger and broader evening peak between 16:00 and 19:00 (probably a reflection of the workday routine), and Tuesday through Thursday carry the highest volumes in the entire dataset. Weekends generally have a different demand pattern: demand starts later, builds gradually through the afternoon, and shows none of the commuter spikes. Across every day of the week, the 00:00–06:00 window remains uniformly low-demand, which makes it the natural slot for charging and maintenance in an electric fleet.

Individual trips are, mostly short urban drives, with a median duration of around ten minutes and running under five miles, alongside a high number of airport and cross-city rides (distributions in Appendix C, Figure 6). The idle time between a driver's consecutive trips lies between 0–30 minute, indicating recurring periods of deliberate waiting or repositioning.

4.2 Spatio-Temporal Analytics

To analyze space consistently, the city is discretized with Uber's H3 hexagonal grid, which offers uniform cell areas and clean adjacency that administrative boundaries lack (grid in Appendix C, Figure 5). The maps below are drawn at resolution 6,7,8, while the forecasting models in Section 5 operate on resolution 5,6,7. This selection is based on avalability of computational resources and the simplicity of manual visual inspection even for high resolutions.

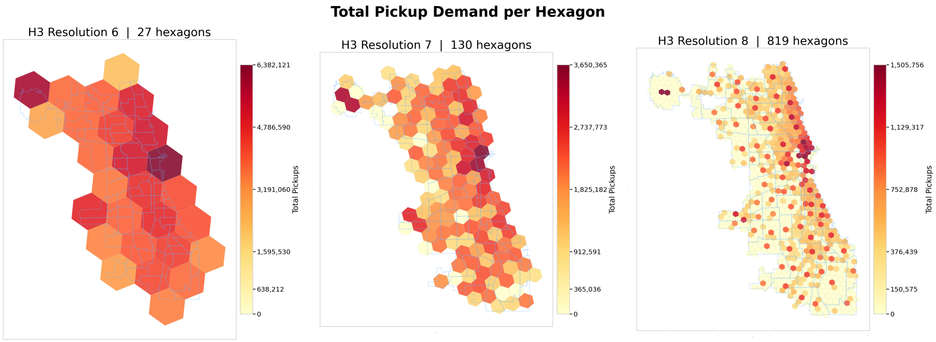

Figure 2: Choropleth maps of pickup demand aggregated at community area level and H3 resolutions 6, 7, and 8.

The Near North Side and the Loop (central pier) clearly dominate demand, consistent with their role as Chicago's central business and entertainment district, whereas O'Hare forms a structurally distinct cluster in the far northwest. As the resolution increases, the hotspots sharpen into specific corridors such as the Magnificent Mile, River North, and the area around O'Hare, and neighborhoods that appear moderate at community level, but vary considerably within themselves. Community-area aggregation therefore conceals precisely the sub-neighborhood differences that a fleet relies on for its positioning decisions.

The picture shifts over the course of the day, yet the downtown-lakefront corridor remains dominant in every 4-hour window (maps in Appendix C, Figure 7). Over night, the downtown core retains its relative activity driven by hospitality traffic. The early mornings add a secondary hotspot around Midway Airport as the first departures begin, and the evening windows intensify the core sharply while the outskirts stay at comparatively low-demand.

4.3 Demand-Flows Between Areas

The origin-destination analysis (matrix in Appendix C, Figure 11) shows the Near North Side and the Loop dominating as both origins and destinations. The O'Hare community area, sends despite its distance, 1,000,000 trips toward those two areas alone. This makes the airport-to-downtown route one of the most dependable sources of paid mileage in the network. Vehicles that drop passengers in the outer neighborhoods, by contrast, face a structural repositioning problem in getting back to high-demand areas. The net flow, defined as pickups minus dropoffs per hexagon, translates pickup-droppoff imbalances into a foundation for optimized work-plans and positioning guidance.

The downtown core, the O'Hare and Midway International Airport consistently have more drop-offs than pickups. The residential outskirts tend to move from an dropoff-overhead towards a pickup-overhead throughout the day. This implies the corresponding working plan to be designed such that ride-hailing vehicles should tend to focus on city center in the early phases of the day (4am - 11am). This strategy should shift, to certain repositioning areas in the north of downtown, as well as the residential outskirts, which get more prevalent in the curse of the day (12am - 11pm). These initial vehicle repositioning strategies are constrained by potential spatio-temporal differences in profitability and trip efficiency. For an electric fleet this has even further implications, with regards to drivers planning, car supply, infrastructure and charging planning, since chargers placed in the net-positive dropoff zones would turn otherwise into idle accumulation, but productive charging can cover the time before potential demand occurs that leads vehicles back to the high-demand areas (details in Appendix C, Figure 8).

4.4 Profitability analysis

The demand map alone would point every vehicle downtown. With the additional perspective of the Demand-Flows residential outskirts in the north among other residential outskirts became more relevant. To introduce a comprehensive view, we add a additional sophisticated analysis layer. We estimate a net profit rate per productive hour for every trip (fare minus a time cost, Chicago's $16.20/hr minimum wage, and a distance cost, EV energy at ComEd rates) and rank hexagons by this rate within each 4-hour window (formula in Appendix C, Equation 1).

Additionally, a we differentiatied by fare per mile and fare per minute, to clearify, which element dominate the profitability. The fare per mile (details in Appendix C, Figure 9) is consistently highest downtown in every time window: short, congested trips still bill close to minimum fare, so revenue per mile stays high as the ride barely moves. Fare per minute (details in Appendix C, Figure 10) shows the opposite geography, running highest on the South Side and near the city's edges, where trips move faster and cover more ground per minute of driving time.

Because profit rate is measured per hour, not per mile, the per-minute effect dominates, which is exactly why downtown, the highest-volume area, is not the highest-profitability area. For the client, this decouples two decisions that look identical on the demand map alone. Downtown is where the volume is, and it is still worth serving for that reason, but a positioning strategy optimized purely for pickup count would systematically under-price driver time relative to other faster-moving areas.

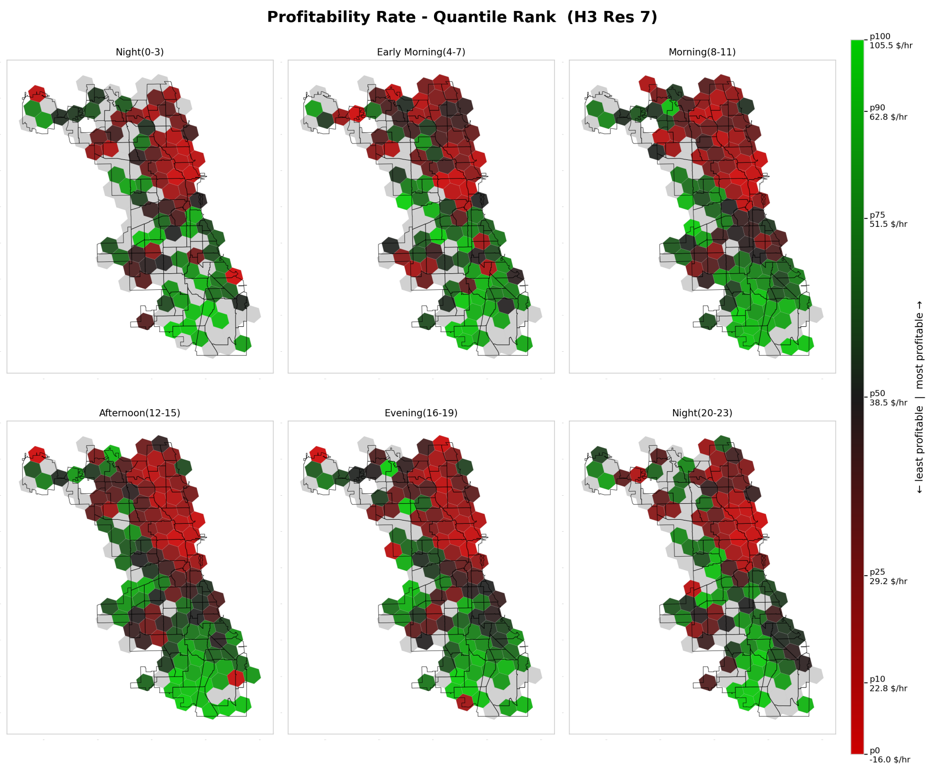

Figure 3: Choropleth maps of profit rate per productive hour, aggregated by hexagon resolutions 7 on 4 hour buckets

The result in the pofitability map is close to a mirror image of the demand map. Downtown is consistently among the least profitable areas per hour, in every time window, while the South

Side and the city's edges rank most profitable. The reason is congestion: downtown trips are short but slow, so a driver earns a high fare per mile but a low fare per minute, and profitability is a per-hour, not a per-mile, measure. These market behaviors have different implications dependent on the scenario the clients plans, concerning charging infrastructure and fleet strength.

The previously created repositioning strategy has to be adjusted in a sense that, if demand occurs throughout the day in the South Side this is preferentially served, because of high profitability. As these trips are rare left over ride-hailing supply should orientate, as previously outlines throughout the day more to the north of downtown.

5 Data Analytics

5.1 Predictive Analytics

5.1.1 Forecasting Problem and Baselines

We set out to predict **taxi demand per hexagon and 4-hour window**, relying solely on information available before the window begins, namely the time of day, day of week, weather forecast, and location. Because a dispatcher scheduling tomorrow's shifts does not yet have tomorrow's actual counts, the models deliberately use no lagged demand. Since trip counts are heavily skewed, with a few downtown cells seeing hundreds of pickups per window while most outer cells see only single digits, we train on log-transformed trip counts and report all metrics back on the trip scale. The grid is split randomly into training, validation, and test data in a 50 / 20 / 30 ratio, and the test set is touched only once, for the final numbers.

To give these results business meaning, we compare them against two naive benchmarks: the *per-cell mean* (the historical average per hexagon and window), scores a MAE of 61.62 trips, while *climatology* (the average per hexagon, hour, and weekday-versus-weekend) scores 23.88. Climatology is the hurdle that really matters, as it is what an operator would already obtain from a simple lookup schedule, and a model therefore justifies its added complexity only by beating it.

5.1.2 Support Vector Regression and Neural Network

We trained two learned models on the same grid and features, so they are directly comparable: a Support Vector Machine for regression and a feedforward neural network. For the SVR, we tuned across kernel types and regularization settings on a validation subsample; the best configuration, an RBF kernel with regularization parameter C = 10, confirms that the link between weather, calendar, and demand is non-linear. For the neural network, a grid search over architectures, learning rates, and optimizers selected a network with two hidden layers of 128 and 64 neurons, trained with the Adam optimizer. Both searches and their intermediate results are in Appendix D, Table 12 and Table 14. Table 2 shows the final test-set results on 41,367 grid cells.

Table 2: Test-set performance of both learned models compared to the naive baselines.

Model	MAE	RMSE	R^2
Per-cell mean	61.62	201.45	0.624
Climatology	23.88	87.86	0.928
SVR (RBF)	21.78	79.74	0.941
Neural network	**18.90**	**72.42**	**0.951**

Both models beat the baselines, but by very different margins. The SVR improves on climatology only modestly, at an MAE of 21.78 against 23.88, or roughly 9 %. Without access to recent actual demand data, most of what it can learn is the seasonal pattern that climatology already encodes, and the 9 % it adds on top comes from the weather and calendar interactions that a fixed schedule cannot represent. The neural network improves on the same baseline even more, at an MAE of 18.90 (21 % reduction), capturing more of the demand structure beyond the weekly rhythm, in particular the way weather, time, and location interact.

Both models share the same weakness: they are accurate in the many low-demand cells but under-predict the busy downtown and airport hexagons where forecasts matter most, a known artifact of training on compressed log counts (decomposition in Appendix D, Table 13). The neural network's residual bias of −5.34 trips shows that the extreme downtown peaks are still partly missed. Those spikes stem from events the data cannot see, such as concerts and sports fixtures, and would require an event calendar to forecast reliably.

Because the granularity of the grid is itself a business decision, we also evaluated both tuned models across coarser and finer grids. The trend is the same for each: finer cells cut the absolute error per cell, simply because each cell holds fewer trips, but the explained variance falls off at the finest resolution, where roughly half the grid is zero-demand (the full sweeps are in Appendix D, Figure 12 and Figure 13). Resolution 6 with 4-hour windows therefore remains the working choice, as moving down to census tracts, roughly 800 units instead of 27, would multiply sparsity without any operational benefit.

5.1.3 Model Comparison

Both models beat climatology, but the neural network improves on it by more than twice the SVR's margin. For an operator positioning vehicles under changing weather or on atypical weekdays, that responsiveness is the point, since a seasonal average cannot say whether tomorrow's rainy Monday will run busier than a dry one. Test error alone does not settle model choice, because a forecast that cannot be explained, retrained, or served cheaply is worth less than its accuracy suggests, and for a taxi fleet the cost of a wrong forecast is asymmetric and lands directly on driver earnings. Table 3 weighs both models and climatology along the dimensions that matter for putting a forecast into production.

Table 3: The forecasting models (H3 resolution 6, 4 hour windows) weighed across production-relevant criteria.

Criterion	Climatology	SVR (RBF)	Neural network
Performance vs. baseline (MAE)	23.88	21.78 (-8 %)	18.90 (-21 %)
Model complexity	Low (lookup table)	Moderate (kernel, few hyperparameters)	High (multi-layer, many weights)
Explainability	Fully transparent	Limited (kernel-implicit feature effects)	Black box
Computational cost (training/storage)	Negligible	High (quadratic in samples), heavy inference	Moderate (near-linear), cheap inference
Maintainability / retraining	Recompute averages	Costly to refit; poor scaling (RBF kernel)	Retrains/fine-tunes incrementally on new data

The three models trade transparency against predictive power in an orderly way. Climatology sits at one extreme. It is a plain lookup table, fully transparent and almost free to compute or refresh, but static by construction and unable to react to weather or atypical days. The SVR occupies the middle on structure yet is the most resource-demanding. Its training cost grows roughly quadratically with the number of samples, which is why it had to be fitted on a subsample rather than the full grid, and its kernel evaluations make prediction heavy too. Refitting it as new months of trip data arrive would inherit that same penalty.

The neural network is the least transparent of the three, effectively a black box, which is the real price of its performance. Against that, it is the more practical model to operate. It trains at near-linear cost in the data, serves each forecast through a handful of cheap matrix multiplications, and, importantly for live dispatch, can be fine-tuned incrementally as fresh trip data accumulates rather than refitted from scratch. So while it wins on raw performance, the more decisive point is that its cost and maintenance profile scales with a growing dataset where the SVR's does not.

Idle time is the criterion that matters most in operation, and it exposes a gap between average accuracy and profitability, because the two error directions are not equally costly. Over-prediction sends drivers to a cell that turns out quiet, where they idle and burn fuel for no fare. Under-prediction at the busy downtown and airport windows is worse, because it leaves the highest-revenue trips unserved while vehicles wait elsewhere. Here the MAE ranking misleads. The neural network has the lower average error but the larger bias at those peaks (-5.34 trips against the SVR's roughly -3), so part of its advantage comes from the many low-demand cells rather than the few that drive earnings. Weighted by revenue rather than by cell count, the two models are closer than the 21 % and 8 % figures imply, and an operator focused on covering peak demand could reasonably prefer the less biased SVR.

The one criterion where the neural network clearly loses is explainability, which matters in fleet operation because a forecast has to earn a driver's trust before it changes their behaviour. A driver sent to a cell against their own instinct, with no reason given, who then finds it empty will lose trust in the tool. For that reason climatology retains value beyond its accuracy. When the neural network deviates sharply from the seasonal norm, that gap is itself a signal worth surfacing to the driver as a reason to move, rather than a black-box instruction to be taken on faith.

5.2 Smart Charging

If forecasting tells the operator where demand will be, charging determines what it costs to serve. To study this, we model a single electric taxi that arrives home at 2 p.m. and must leave the garage again at 4 p.m. Rather than charging at a flat rate, an agent sets the charging power every 15 minutes, balancing two competing goals: keeping the recharging cost low and guaranteeing enough energy for the upcoming shift, whose demand is still unknown at the time the charging decisions are made. Because these decisions are sequential and the outcome uncertain, we frame the problem as one of reinforcement learning and train a Deep Q-Network (DQN). The agent observes the battery's state of charge (SoC) and the current time slot, chooses among four power levels (off, 7.3, 14.7, or 22 kW), pays a charging cost that grows steeply with power, and incurs a large penalty if the battery falls short of the realized shift demand. The full environment design, agent setup, and training diagnostics are provided in Appendix E.

The learned policy (Appendix E, Figure 16) is simple enough to hand to an operations team, amounting to charging hard while the battery is low, tapering off as a buffer builds, and switching off once the SoC comfortably exceeds expected demand. Because a shortfall is penalized far more heavily than expensive charging, the policy is risk-averse by construction and targets a small surplus above the 30 kWh mean demand rather than the mean itself.

Table 4: Policy comparison over 1,000 evaluation episodes.

Policy	Avg. Reward	Avg. Cost	Avg. Penalty	Avg. Final SoC (kWh)	Shortfall rate
DQN Agent	**-65.10**	64.45	0.65	42.8	**0.4 %**
Always-Max	-86.99	**86.99**	0.00	56.5	0.0 %
Random	-202.91	56.39	146.52	34.4	29.1 %

Evaluated over 1,000 episodes (Table 4), the agent delivers nearly the safety of always charging at maximum while achieving a far better cost profile. Always-Max never runs short but pays roughly 35 % more in charging cost for a buffer it does not need, ending with an average of 56.5 kWh against a 30 kWh mean demand, and the random policy illustrates the opposite failure mode, since its 29 % shortfall rate would mean roughly one in three shifts starting under-charged. The DQN agent, by contrast, runs short in only 0.4 % of episodes, and this reliability level can itself be tuned through the shortfall penalty. The trade-off between cost and reliability is visualized in Figure 4.

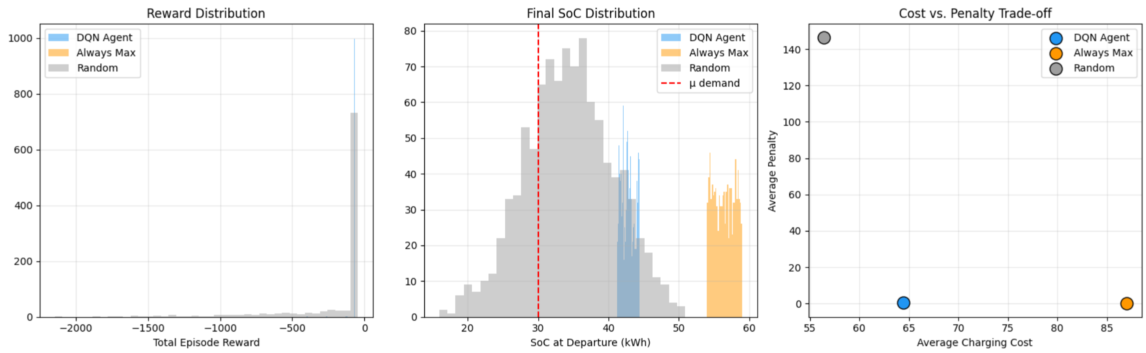

Figure 4: Distributions over 1,000 evaluation episodes: total reward, final SoC, and cost vs. penalty, compared across DQN Agent, Always-Max, and Random.

For the client, the key takeaway is that "charge to full" is not cost-optimal even though it feels safest. Since the state space is only two-dimensional, the learned policy could be deployed as a plain lookup table on a home charging unit. Two caveats nonetheless belong in the client conversation: the agent is trained on a fixed demand distribution and would need retraining if trip volumes shift, and the shortfall penalty is a dial to be set according to the operator's actual tolerance for an under-charged vehicle rather than a fixed technical constant.

6 Conclusions and Recommendations

6.1 What the Analysis Shows

Demand in Chicago is concentrated in space and structured in time. A handful of hexagons, covering the downtown core, the Magnificent Mile, and the O'Hare corridor, account for a disproportionate share of all pickups, while most of the city sits close to zero for much of the day. This spatial concentration is overlaid by a predictable weekly rhythm: bimodal weekdays with a strong evening peak, later and flatter weekends, and a quiet overnight window common to every day.

Both forecasting models beat the seasonal climatology baseline, the neural network by a clear margin (MAE 18.90 against 23.88) and the SVM more modestly (21.78 against 23.88). The reinforcement learning agent, in turn, learned a charging policy that reliably met the driver's departure energy needs at a cost well below a naive charge-at-maximum strategy. For BWP, these three findings map directly onto the launch decisions raised in the introduction: where to enter the market, how far to trust a forecast over a fixed schedule, and how to power the fleet without eroding margins.

6.2 Recommendations for the Fleet Operator

Position vehicles for the corridors, not the map as a whole. Because demand lives in a handful of places, BWP should phase the rollout rather than spread it evenly across all 77 community areas, beginning with the downtown-to-lakefront corridor and the airport connections. O'Hare alone sends close to half a million trips toward the Near North Side and the Loop, which makes the airport-to-downtown axis a dependable source of paid mileage from day one and a low-risk place to concentrate the initial fleet.

Use the net-flow signal to plan repositioning. Pickups and dropoffs do not balance locally. Downtown and the airport hand out more trips than they take back, whereas the residential edges absorb vehicles without generating return demand, so a fleet that ignores this pattern gradually accumulates idle cars in the wrong places. Repositioning logic should therefore be built around the net-flow map, treating the outer sinks as places to hold and charge rather than places to wait for fares.

Forecast for planning, and know what the forecast cannot see. The neural network earns its added complexity because it responds to weather and atypical days in a way a fixed schedule cannot, which makes it well suited to shift planning and to questions such as expected demand in a given area on a rainy Sunday afternoon. What it does not capture are one-off spikes driven by concerts, fixtures, and conventions. Its output is best treated as a strong baseline to be overridden by hand on known event days, not as a complete demand picture. In practice, BWP can run the neural network as the default engine for driver scheduling and let dispatchers layer local event knowledge on top.

6.3 Private or Public Charging

Taken together, the charging economics and the demand geography point to a mixed answer. Because the cost function penalizes high charging power convexly, charging slowly over a long window is much cheaper than drawing short bursts at the full rate. The agent exploited exactly this, spreading energy across the available slots and holding a modest buffer above expected demand. Such a strategy favors private or depot charging for the bulk of the fleet's energy, since it lines up with the quiet overnight window, draws at a low and cheap rate for hours, and can be located in the negative-net-flow residential zones where cars would otherwise idle.

Public fast charging works against this cost structure, as its entire value lies in delivering energy quickly, which is precisely the expensive corner of the curve. We therefore recommend that BWP anchor the fleet on private or depot charging near where vehicles naturally end their shifts, and reserve public fast charging as an opportunistic top-up for a car that would otherwise have to leave a high-demand corridor to recharge. For infrastructure planning, this means investing first in depot capacity in the residential sink zones rather than in a broad network of fast chargers.

6.4 Limitations

Several caveats bound our findings. Locations are reported at census-tract centroids rather than precise locations, so hotspot detection and flow maps inherit a coarse spatial grain and fine within-tract patterns remain invisible by construction. Furthermore, the forecasting models see neither recent demand counts nor an event calendar, which explains why both under-predict the largest downtown spikes and why the SVR only modestly improves on a seasonal average. The SVR also had to be tuned on a small subsample, as computational complexity increases significantly with the full grid. The charging agent, finally, is trained against a stylized environment with a fixed cost coefficient and normally distributed departure demand. Because real prices and energy needs vary, the learned policy should be read as a proof of concept rather than a deployable controller. BWP should therefore treat the numbers above as directional evidence for where and how to launch, and validate them against live operating data once the first vehicles are on the road.

6.5 Outlook

For BWP, the most valuable next step is adding lagged demand to the forecasts. The previous period's actual count is usually the strongest single predictor, and its absence is the clearest gap between this setup and a production system. Beyond it, a spatially aware hexagon encoding (distance to center, proximity to transport hubs, neighboring demand) would help the models generalize across cells, while an event calendar would target precisely the spikes they currently miss. On the charging side, real day-ahead electricity prices would let the agent optimize against the tariffs the fleet actually faces. Further ahead, pairing the demand maps with fare-per-mile and idle-time patterns would reveal where drivers earn most efficiently, and an explicit repositioning model would translate the net-flow findings into concrete vehicle movements. Together these steps form a practical roadmap for turning this initial study into the operational decision engine behind BWP's Chicago launch.

Appendix

A Data Dictionaries

A.1 Taxi Trip Data

Table 5 lists the key fields of the trip dataset. Seven temporal features (date, hour, day of week, month, week, weekend flag, holiday flag) are derived at load time, bringing the working total to 28 columns.

Table 5: Core fields of the Chicago taxi trip dataset.

Column	Type	Description
`trip_id`	string	SHA-1 hash; unique journey identifier
`taxi_id`	string	SHA-1 hash; anonymized vehicle identifier
`trip_start_timestamp`	datetime (UTC−6)	Rounded to the nearest 15 minutes
`trip_end_timestamp`	datetime (UTC−6)	Rounded to the nearest 15 minutes
`trip_seconds`	float	Recorded journey duration in seconds
`trip_miles`	float	Recorded journey distance in miles
`pickup_census_tract`	float	Census tract of pickup ($\approx 54\%$ missing)
`dropoff_census_tract`	float	Census tract of dropoff ($\approx 56\%$ missing)
`pickup_community_area`	float	Community area of pickup (0.05% missing)
`dropoff_community_area`	float	Community area of dropoff (7.2% missing)
`fare`	float	Metered fare in USD
`tips`	float	Tip amount in USD
`trip_total`	float	Total charge billed to the passenger
`payment_type`	string	Credit card, cash, mobile, etc.
`company`	string	Taxi company name
`pickup_centroid_lat/lon`	float	Centroid of the pickup census tract
`dropoff_centroid_lat/lon`	float	Centroid of the dropoff census tract

A.2 Weather Data

The three weather zones are centered on the downtown core (41.897°N, 87.639°W), the O'Hare area (41.980°N, 87.906°W), and the South Side (41.769°N, 87.656°W). Both the elbow method and the silhouette score pointed to k = 3. Table 6 lists the hourly variables fetched for each zone.

Table 6: Hourly weather variables fetched from Open-Meteo.

Variable	Unit	Description
`temperature_2m`	°C	Air temperature at 2 m height
`apparent_temperature`	°C	Feels-like temperature (wind chill / heat index)
`precipitation`	mm	Total precipitation
`rain`	mm	Liquid precipitation
`snowfall`	cm	Snowfall accumulation
`snow_depth`	cm	Snow depth on ground
`windspeed_10m`	km/h	Wind speed at 10 m
`windgusts_10m`	km/h	Wind gust speed at 10 m
`cloud_cover`	%	Fractional cloud cover

A.3 Point-of-Interest Data

The OSM query covers four tag categories (`amenity`, `shop`, `tourism`, `leisure`). Raw tags are collapsed into seven demand-relevant categories via a lookup table, grouping tags with similar expected demand timing and dropping types too niche to matter (Table 7).

Table 7: POI counts by functional category (OpenStreetMap, Chicago).

Category	Count	Examples
Food & Nightlife	5,652	Restaurants, bars, cafés, fast food
Education	1,220	Schools, universities, libraries
Shopping	1,085	Supermarkets, clothing, electronics
Healthcare	469	Hospitals, clinics, pharmacies
Entertainment	350	Theaters, cinemas, museums
Accommodation	231	Hotels, hostels
Transport	134	Bus stops, subway entrances, ferry terminals

B Data Preparation Detail

B.1 Missing Values

Different fields are missing for different reasons, so each gets its own treatment (Table 8).

Table 8: Missing-value handling by field.

Field	Missing	Cause	Treatment
Timestamps	< 0.01 %	Fall in the autumn DST fold, where one hour repeats and the occurrence is ambiguous	Remove

Field	Missing	Cause	Treatment
Drop-off community area	7.2 %	Drop-off outside the 77 official community areas (suburban)	Recover via spatial join where inside the city; retain suburban trips unassigned
Trip duration	small	Not reported but derivable	Impute from end − start timestamp
Trip distance	small	Not reported	Estimate with great-circle distance (understates road distance)
Vehicle ID	8 records	Unattributable to any taxi	Remove

B.2 Consistency Checks

Some records are present but aren't real journeys. **Ghost trips** have zero seconds, identical start and end time, and identical start and end location, which looks like a meter switched on and off. All four conditions must hold, so genuine short trips survive; we remove the **152,967** records that qualify (~1 %). A further **1,013,003 trips** (6.8 %) log zero distance despite positive duration, with a median of 73 seconds and a quarter under 7 seconds. These are overwhelmingly meter cancellations or GPS failures rather than real journeys, and all are excluded. We also confirmed there are no end-before-start inversions and no duplicate trip IDs.

B.3 Outlier Removal

After dropping non-journeys, we remove physically implausible values using two derived metrics, average speed and fare per mile (Table 9).

Table 9: Outlier removal criteria applied to the taxi trip dataset; the union of all rules drops 269,407 records (1.96 %).

Criterion	Records flagged	Share of dataset
Non-positive fare	5,252	0.04 %
Distance exceeding 100 miles	1,011	0.01 %
Duration exceeding 3 hours	12,651	0.09 %
Average speed above 70 mph	6,600	0.05 %
Average speed below 2 mph	155,659	1.13 %
Fare below \$2 per mile	28,567	0.21 %
Fare above \$100 per mile	115,730	0.84 %

B.4 Cleaning Levels

Table 10: Rows retained under each cleaning level.

Level	What is removed	Rows retained
Demand (minimal)	DST-ambiguous timestamps	14,984,551 (99.99 %)

Level	What is removed	Rows retained
Trip (full)	Above, plus missing vehicle ID, ghost trips, zero-distance trips, and speed/fare outliers	13,485,728 (89.99 %)

B.5 Weather and POI Preparation

Condition codes. The raw weather data carries a WMO condition code (0–99). Treating it as a number implies a false ordering (code 95 isn't "95× worse" than code 1), and only three conditions actually appear: clear, rain, snow. Since rain and snow already have their own numeric columns, the code adds nothing and is dropped.

Clock-change gaps. Each zone shows a one-hour gap on spring-forward days (2024-03-10, 2025-03-09, 2026-03-08). The 02:00 hour doesn't exist when clocks jump 01:00 → 03:00, so the gap is correct and no imputation is needed.

POI geometry. The OSM query returns points, polygons, and lines. All non-point features are reduced to their centroid, so every POI is a single coordinate. Each is mapped to one of seven functional categories via a lookup table, with unmapped types discarded; the final 9,141 POIs are assigned to the hexagon containing their coordinate.

B.6 Granularity and Sparsity

Grain sets a trade-off: finer units capture local variation but leave more cells at or near zero, which is harder to forecast. Table 11 shows the effect.

Table 11: Zero-demand share across spatial and temporal granularities.

Spatial unit	Time bucket	Grid cells	Zero-demand share
H3 resolution 5 (large hexagons)	1 hour	163,400	5.8 %
H3 resolution 6	1 hour	551,475	26.8 %
H3 resolution 6	4 hours	137,889	12.6 %
H3 resolution 7	4 hours	602,626	49.2 %

C Additional Descriptive Figures

$$\text{profit\_rate}_i = \frac{\text{fare}_i - C_{\text{time}} \cdot (t_i^{\text{trip}} + t_i^{\text{idle}}) - C_{\text{dist}} \cdot d_i}{t_i^{\text{trip}}/3600} \tag{1}$$

D Model Tuning and Diagnostics

D.1 SVR Kernel and Hyperparameter Search

A Support Vector Machine for regression (SVR) fits a function that stays within a tolerance band around the observed values while remaining as smooth as possible. The kernel sets the shape of that function: linear allows only straight-line relationships, polynomial curved ones, and the Radial Basis Function (RBF) highly flexible non-linear relationships. We start simple and add complexity, so that any gain can be attributed to the added flexibility rather than assumed.

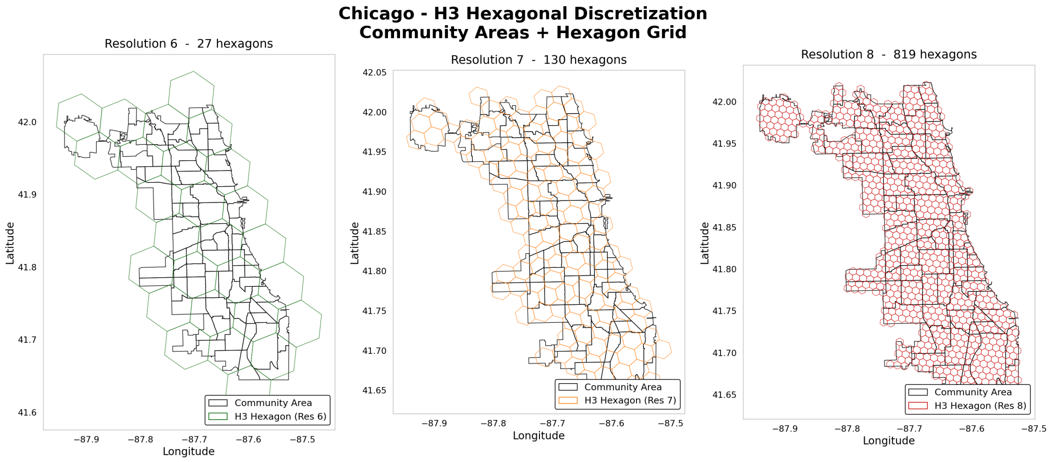

Figure 5: H3 hexagonal grid overlaid on Chicago community areas at resolutions 6, 7, and 8.

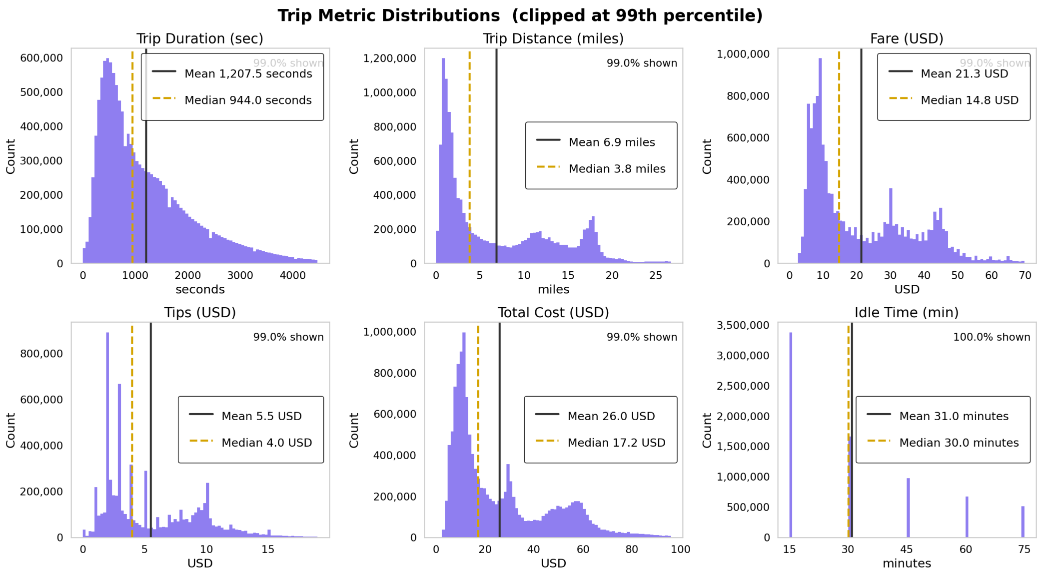

Figure 6: Distribution of trip duration, distance, fare, tips, total cost, and inter-trip idle time across the full dataset. Distributions are clipped at the 99th percentile so airport flat rates and exceptional idle gaps don't distort the axes.

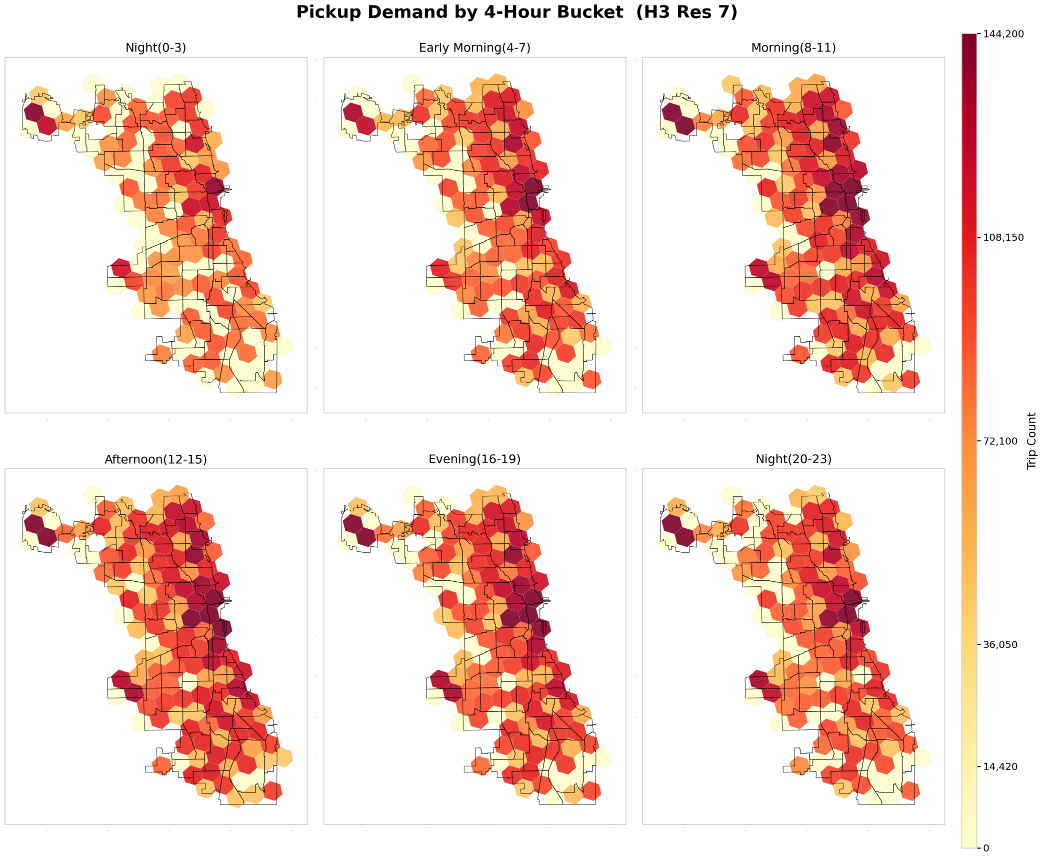

Figure 7: Pickup demand by 4-hour time bucket, covering the full 24-hour cycle.

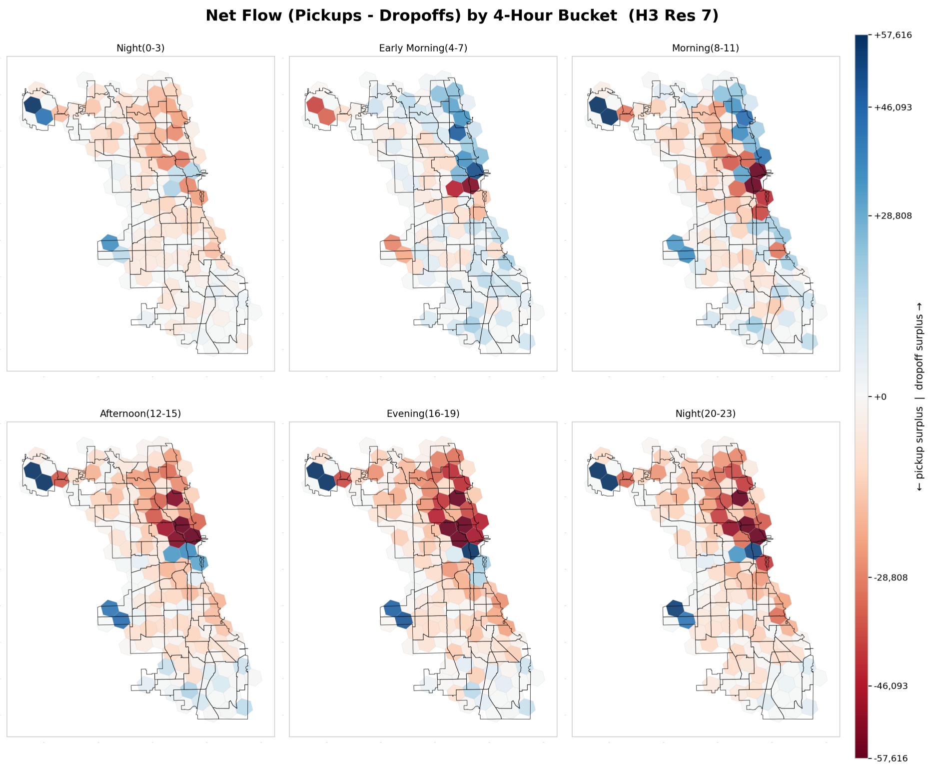

Figure 8: Net flow (pickups minus dropoffs) by H3 hexagon, aggregated across the full observation period.

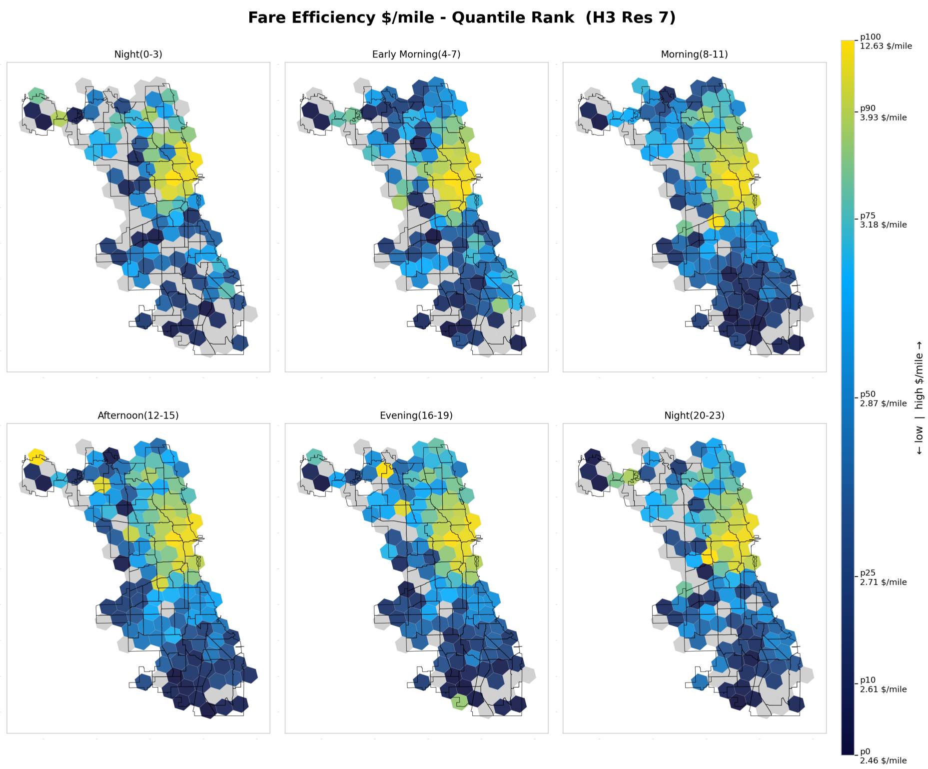

Figure 9: Choropleth maps of fare/mile, aggregated by hexagon resolutions 7 on 4 hour buckets

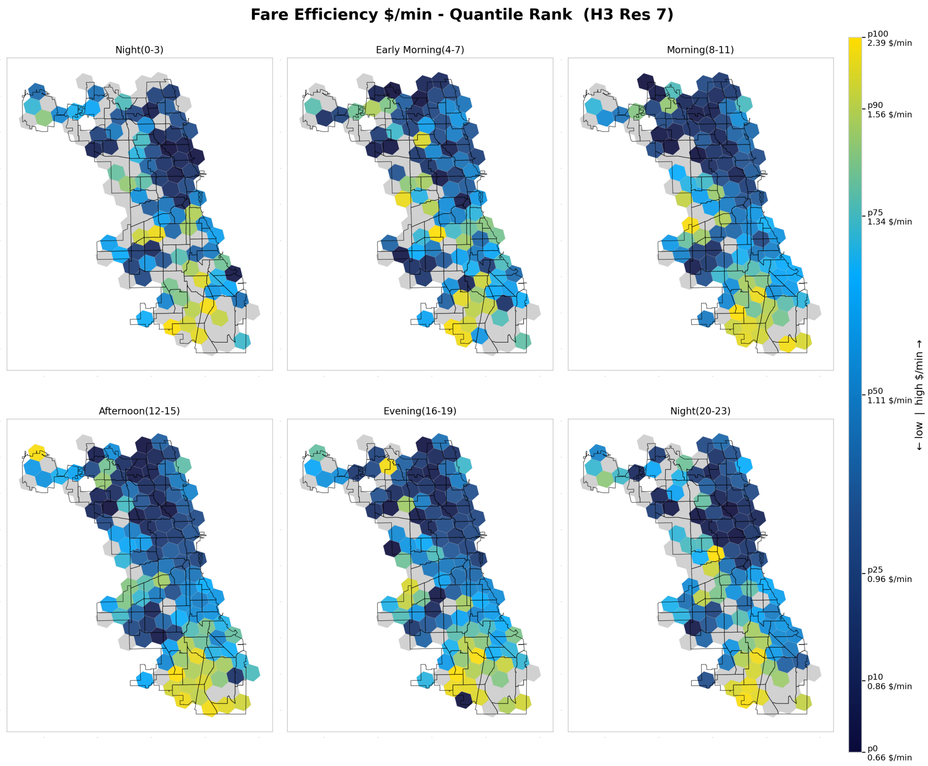

Figure 10: Choropleth maps of fare/min, aggregated by hexagon resolutions 7 on 4 hour buckets

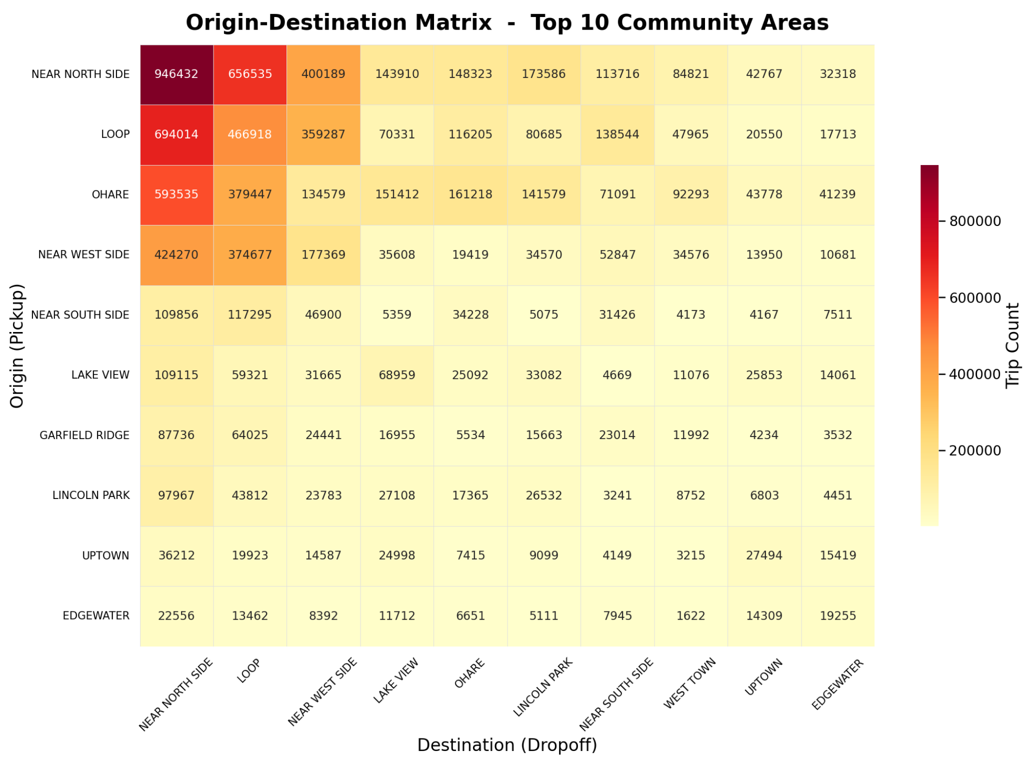

Figure 11: Origin–destination matrix for the top 10 community areas by total trip volume.

Three settings are tuned alongside the kernel: C, how closely the model fits the training data; epsilon, the width of the no-penalty tolerance band; and gamma, how far each observation's influence reaches (RBF and polynomial only). SVR scales poorly with training set size, so tuning ran on a random subsample of 8,000 observations, with the best configuration refit on 30,000 rows for final evaluation.

Table 12: Best SVR configuration per kernel type, scored on the validation set.

Kernel type	Best regularization	Validation MAE
Linear	C = 0.1	36.96 trips
Polynomial	C = 10.0	27.44 trips
RBF	C = 10.0	**23.70 trips**

Moving from linear (MAE 36.96) to RBF (MAE 23.70) shows the value of flexibility: the link between weather, calendar and demand is non-linear, and the RBF kernel captures that curvature. The RBF kernel with C = 10 and epsilon = 0.1 goes into final evaluation.

D.2 SVR Error Decomposition

Table 13: SVR test-set error decomposed by demand level.

Demand level (trips per window)	MAE	Observations
0–1	0.72	8,373
1–5	1.69	7,417
5–20	3.57	10,232
20–100	11.86	8,406
100+	107.54	6,939

D.3 Performance Across Resolutions

We re-evaluated each tuned model across coarser and finer H3 grids (resolutions 5, 6, and 7) and time windows (1, 4, and 8 hours). Figure 12 and Figure 13 report R^2 and MAPE (over non-zero-demand cells) for the SVR and the neural network respectively.

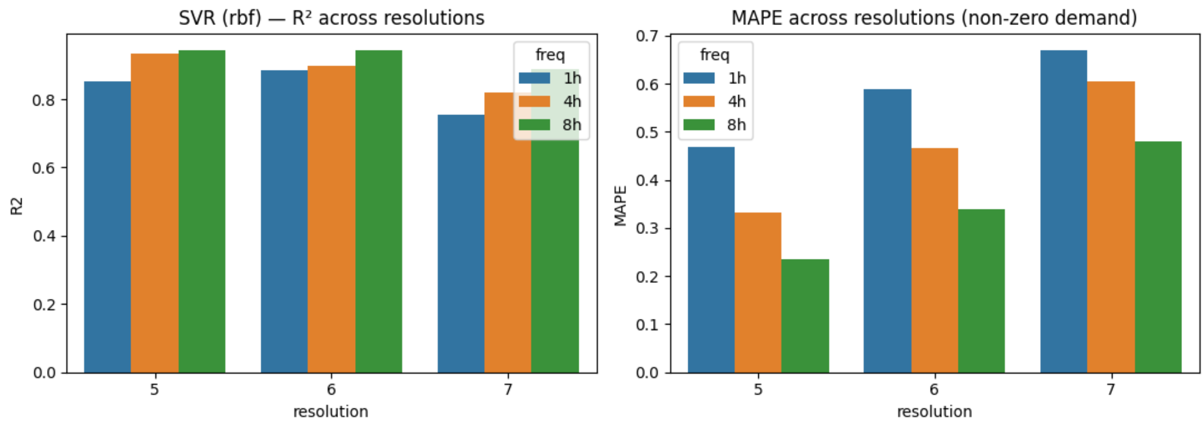

Figure 12: SVR (RBF) performance across spatial resolutions and time windows.

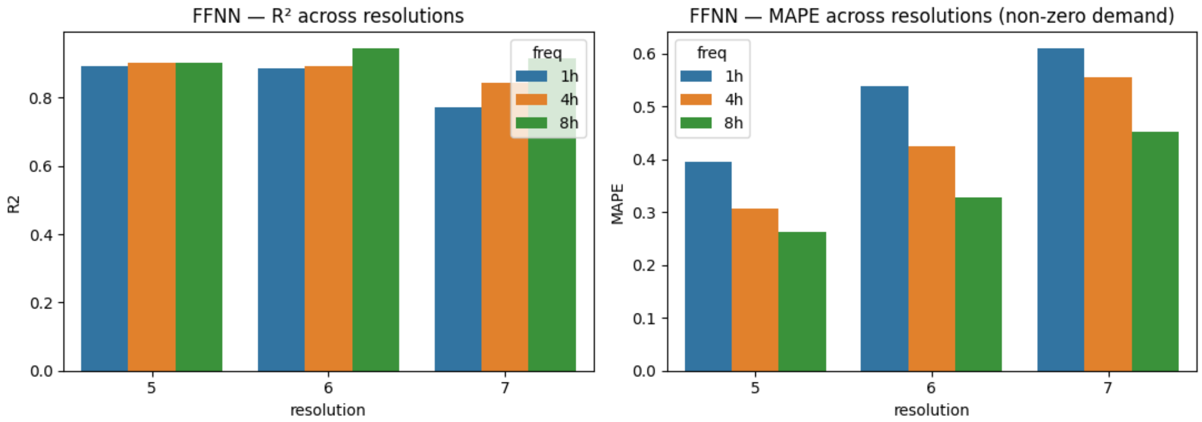

Figure 13: Neural network performance across spatial resolutions and time windows.

Both models follow the same pattern. MAPE falls as the window widens, since longer windows aggregate more trips per cell and smooth out per-cell noise. R^2 is highest at resolutions 5 and 6 and drops at resolution 7, where roughly half the grid is zero-demand and the per-cell signal gets noisy. Resolution 6 with 4-hour windows sits at a favorable corner of both surfaces for either model, which is why it is used throughout.

D.4 Neural Network Search and Diagnostics

The grid search covers hidden layer sizes ([64, 64], [128, 64], [256, 256] and [64, 64, 64, 64]), dropout (0 %, 10 %), learning rate (0.1, 0.01, 0.001) and optimizer (SGD, Adam). Each candidate trains for up to 200 iterations, stopping early after 20 iterations without validation improvement. The winning configuration (two hidden layers of 128 and 64 neurons, no dropout, learning rate 0.001, Adam) is retrained from scratch on the combined training and validation data.

Holding that architecture fixed, we compare three activation functions (Table 14). Tanh edges out ReLU, and both are far ahead of sigmoid, which never trains to a usable error level because of gradient saturation: its neurons produce near-constant outputs across much of the input range, which stalls learning in deeper networks. The final model uses ReLU; the gap to tanh is under two trips of MAE, small enough to fall within run-to-run variation on a single validation split.

Table 14: Validation error by activation function for the best network architecture.

Activation function	Validation MAE
ReLU	17.86 trips
Tanh	**17.50 trips**
Sigmoid	81.18 trips

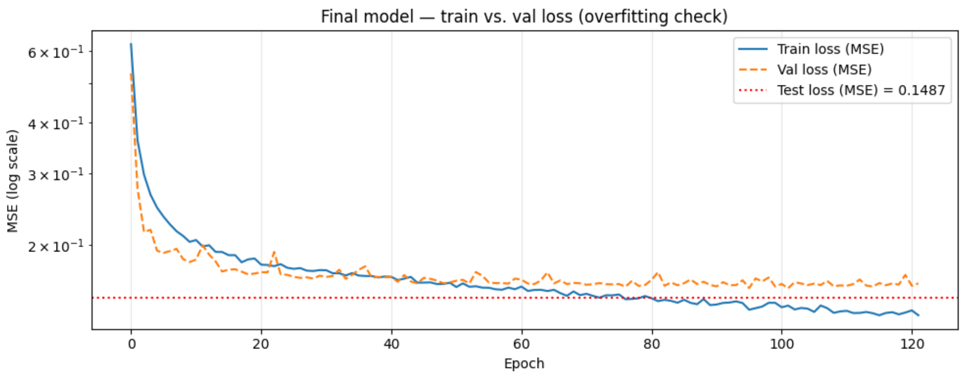

Figure 14: Training and validation loss curves for the final neural network model.

E Smart Charging Environment and Training

E.1 Markov Decision Process Design

State space. The state is a 2-dimensional vector: the current SoC in kWh, and the current 15-minute slot index, normalized to $[0, 1]$. Two components are sufficient because the charging window is fixed (2 hours, 8 slots) and the only quantity that evolves within it is the battery level; calendar or weather features would be redundant, since none of them change within a single 2-hour window. Keeping the state low-dimensional also keeps the learned policy interpretable as a simple heatmap over SoC and time slot (Figure 16), which matters for explaining the agent's behavior to the client.

Action space. Charging power is discretized into four levels (off, low, medium, high: 0, 7.3, 14.7, 22 kW) rather than allowing continuous output. The assignment explicitly permits this simplification, and discrete actions keep DQN applicable without the added complexity of a continuous-control algorithm such as DDPG, which would need substantially more tuning for a 4-way decision that a home charging unit would realistically only support in discrete steps anyway.

Reward function. Every slot incurs a charging cost that grows exponentially in power, $c \cdot e^{p_t/p_{\max}}$, reflecting that fast charging draws a disproportionate premium on real electricity tariffs (modeled here on average ComEd afternoon supply prices) and stresses the battery more than slow charging. At the end of the window, a shortfall between the realized stochastic demand ($\mathcal{N}(30, 5^2)$ kWh) and the final SoC is penalized at a coefficient of 100 per kWh missing, deliberately far above the cost scale of charging itself, so that the agent never learns to gamble on running out of energy to save a few cents. Table 15 summarizes the resulting reward per action, evaluated at the reference cost coefficient of the simulation.

Table 15: Charging cost by action, before any shortfall penalty.

Action	Label	Power (kW)	Energy per slot (kWh)	Cost this slot
0	Off	0.0	0.00	4.00
1	Low	7.3	1.82	5.58
2	Medium	14.7	3.67	7.80
3	High	22.0	5.50	10.87

A note on scaling. An earlier version of the cost function used $c \cdot e^{p_t}$ directly. Because p_t ranges up to 22, this exploded numerically and made the reward signal uninformative for the agent. Normalizing the exponent by $p_{\max}$ keeps the cost in a well-behaved range while preserving the intended convexity: charging faster remains disproportionately expensive, just not to the point of destabilizing training.

E.2 Agent: Deep Q-Network

We use a Deep Q-Network rather than tabular Q-learning or policy-gradient methods, for three reasons tied to this problem's structure. First, the action space is small and discrete (four choices), which is exactly the regime Q-learning-based methods are strongest in; policy-gradient methods (REINFORCE, Actor-Critic) add complexity that buys nothing here. Second, the state space is continuous (SoC is a real-valued kWh amount), so tabular Q-learning would require discretizing it and lose precision at the SoC boundaries near the shortfall threshold. Third, DQN's experience replay lets the agent learn from individual 15-minute transitions rather than waiting for full episodes to complete, which is more sample-efficient than Monte Carlo approaches given the small number of decision points (8) per episode.

The Q-network is a small feedforward network with two hidden layers (12,932 parameters total), trained with a target network and soft updates ($\tau = 0.0005$) for stability, and an ε-greedy exploration schedule decaying from 1.0 to 0.001 over training. Training ran for 5,000 episodes (40,000 transitions).

E.3 Training Behavior

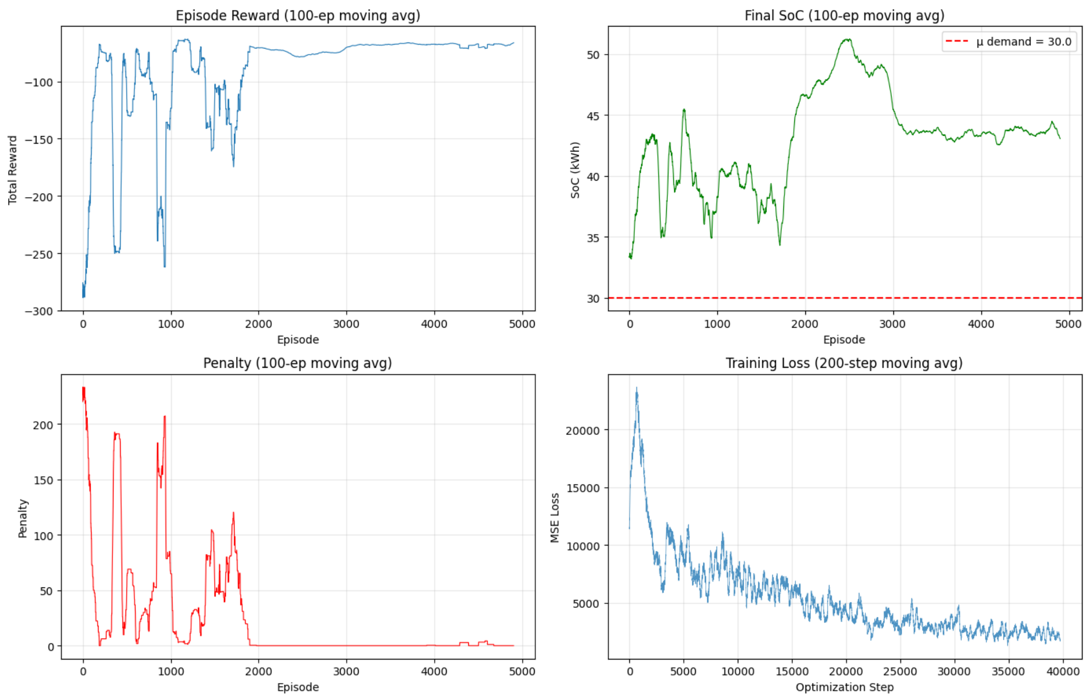

Figure 15: Training curves: episode reward, final SoC, cost and penalty over 5,000 training episodes (100-episode moving average).

Figure 15 shows the training curves. Reward and shortfall penalty do not decline monotonically: several stretches (around episodes 500 and 1,000) show a spike in average penalty, where the agent temporarily under-charges before correcting. This reflects the exploration-driven trade-off inherent to ε-greedy DQN. The agent must occasionally under-charge to learn the shape of the penalty, since the shortfall penalty is only observed at the end of an episode. By the later episodes, the average penalty converges to zero while average final SoC stabilizes above the 30 kWh demand mean, indicating the agent has learned a conservative buffer against demand uncertainty rather than charging to the mean exactly.

E.4 Charging Policy

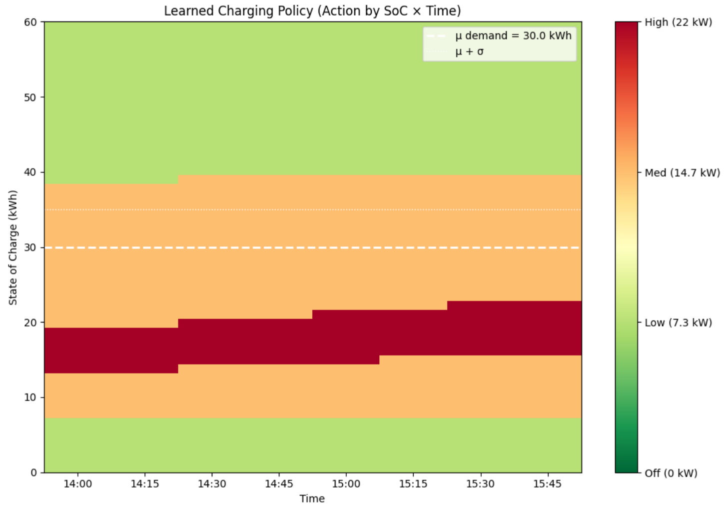

Figure 16: Learned charging policy across SoC and time slot: preferred action (off / low / medium / high) per state.

E.5 Example Episode

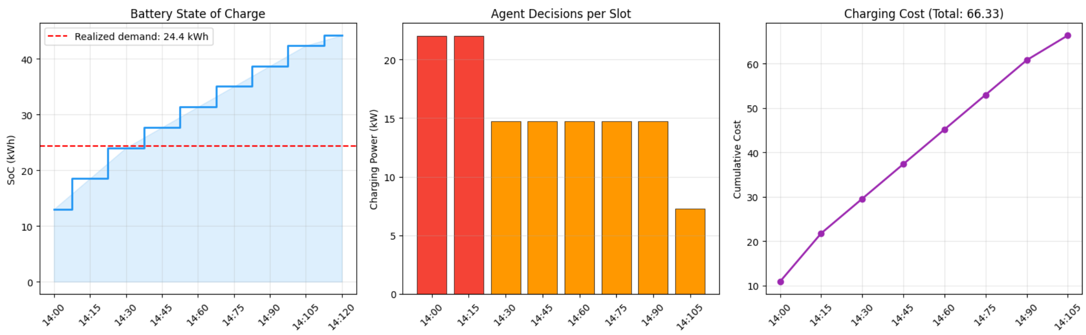

Figure 17: Example evaluation episode: SoC, charging power, and cost per 15-minute slot from 2 p.m. to 4 p.m.

Figure 17 traces one evaluation episode slot by slot: SoC rises from an initial value in the 10–15 kWh range at 2 p.m. to a final SoC of 44.2 kWh at 4 p.m., against a realized demand draw of 24.4 kWh, leaving a comfortable 19.8 kWh surplus at a total charging cost of 66.33. This single trace should not be read as typical (the realized demand here happened to fall below the mean), but it illustrates the slot-by-slot decision pattern: higher power early in the window while SoC is low, tapering off as the buffer builds.

References

Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, *65*, 126–139. https://doi.org/10.1016/j.compenvurbsys.2017.05.004

Open-Meteo. (2024). Historical weather API [Accessed: 2026-05-10].